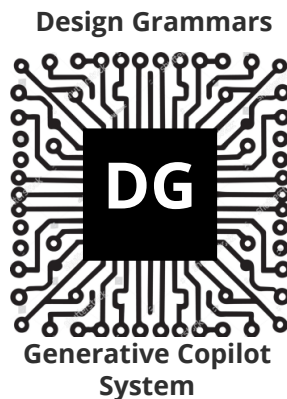


MANAGEMENT OF THE DESIGN KNOWLEDGE CORE IN A CROSS-PLATFORM ENVIRONMENT OF THE BIM PROJECT

Carrying parametric design intent and machine-readable design rules across the openBIM lifecycle.



Speaker: Evgenii Ermolenko ⁽¹⁾
Supervisors: Bruno Figueiredo ⁽²⁾
Miguel Azenha ⁽³⁾

(1) University of Minho, EAAD, Lab2PT, ISISE, Guimarães, Portugal, 0009-0003-1228-9550

(2) Universidade do Minho, EAAD, Lab2PT, IN2PAST, Guimarães, Portugal, 0000-0001-8439-7065

(3) University of Minho, ISISE, ARISE, Department of Civil Engineering, Guimarães, Portugal, 0000-0003-1374-9427



Universidade do Minho
Escola de Engenharia

Universidade do Minho
Escola de Arquitetura, Arte e Design



Laboratório de Paisagens,
Património e Território



Fundação
para a Ciência
e a Tecnologia



This research was funded by Minho University and Lab2PT (Landscapes, Heritage, and Territory Laboratory) through a Doctoral Grant from the Foundation for Science and Technology (FCT), reference number 2024.03763.BD.

MANAGEMENT OF THE DESIGN KNOWLEDGE CORE IN A CROSS-PLATFORM ENVIRONMENT OF THE BIM PROJECT

Table of Contents

- 1. Design Knowledge**
- 2. Design State**
- 3. The Knowledge Gap**
- 4. Proposed system: The Design Grammars. Goal and Approach**
- 5. Positioning — relation to IFC, IDS and ISO 19650**
- 6. System Framework**
- 7. DG Ontology**
- 8. DG System Prototype. Design State validation**
- 9. Conclusion**

DESIGN KNOWLEDGE

THE LOGIC BEHIND THE MODEL



Carrying parametric design intent and machine-readable design rules across the openBIM lifecycle.

Design knowledge is the explicit, machine-readable body of functions, constraints, rules and generative relationships that justify and produce a design — its logic and intent — as distinct from the realised building objects, their relationships and properties that a BIM/IFC model records as the finished state.

FBS (Function-Behavior-Structure) framework ⁽¹⁾

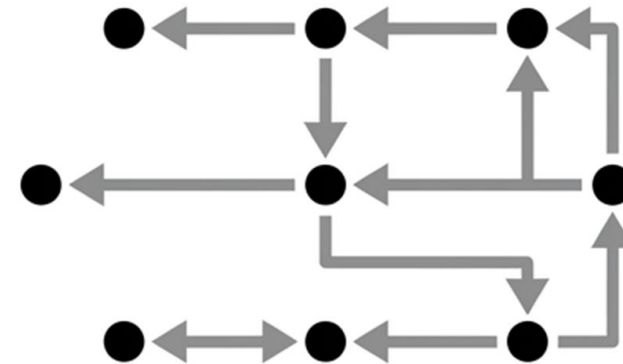
The Realised What Structure



BIM / IFC — the object structure of specific design state.

Describes the building as **objects, their relationships and properties** — IfcSpace, IfcWall, IfcElement, spatial containment, aggregation, connectivity and attributes.

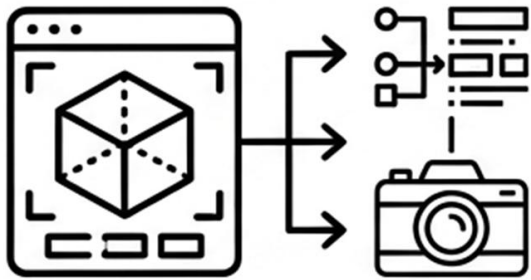
The Why and How Function-Behavior



Design Knowledge — the *why* and *how* upstream (Function + Logic)

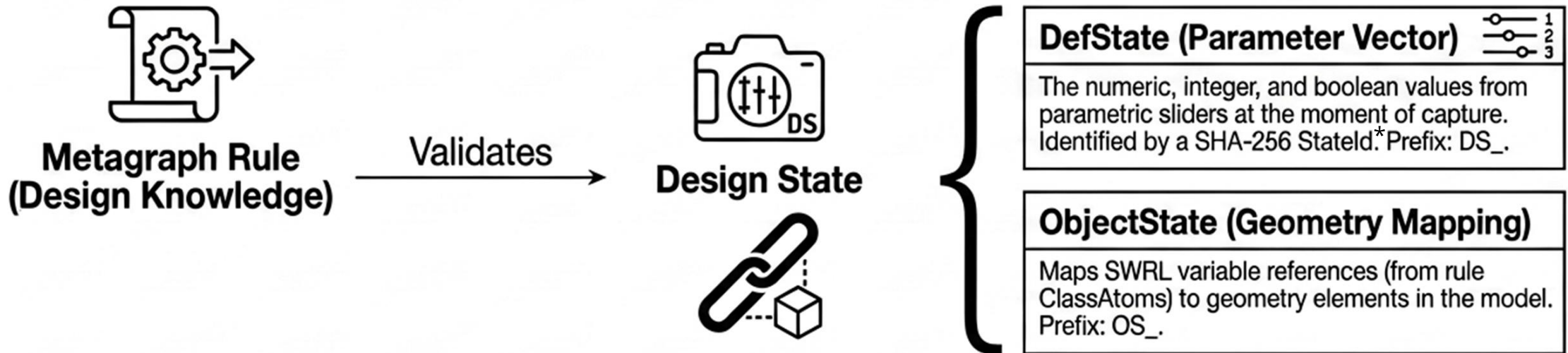
Functional intent, prescriptive rules and constraints a valid design must satisfy, parametric/generative logic driving form, and the rationale linking intent to geometry.

*FBS framing ⁽¹⁾: IFC carries the building's **Structure** — objects, relationships, properties. Design Grammars carries its **Design Knowledge** — Function, rules and generative logic that produced it.*



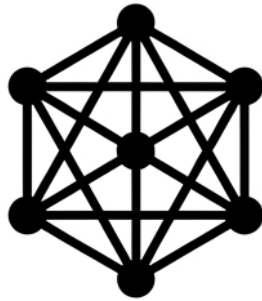
THE DESIGN STATE

Uniquely identified snapshot of a design's current parametric configuration — the set of input parameters and object–geometry assignments captured from the parametric tool at a specific point in time.



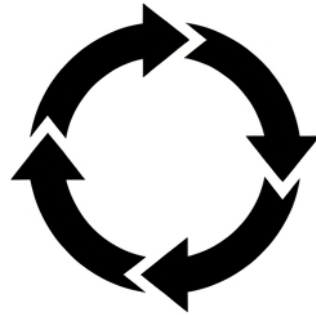
* **SHA-256** - FIPS PUB 180-4 standard (NIST, 2015), part of the SHA-2 family of cryptographic hash functions.

The Three Pillars of Design Knowledge



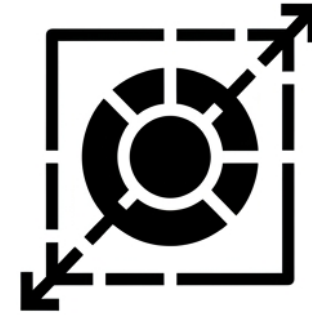
Explicit & Machine-Readable

Encoded as formal ontology and SWRL atoms. Not implicit in a VPE* or locked in a designer's head.



Generative & Prescriptive

It actively produces and tests design iterations against constraints, rather than passively describing a finished one.



Tool- & File-Independent

It lives in a shared semantic graph core, establishing object-level communication across platforms rather than file-based exports.

* VPE – Visual Programming Environment

How much design knowledge survives the next handover?

ISO 19650 governs **what** information is delivered — **never the design logic that produced it.**

From Concept Design to Maintenance – stripping away Design Knowledge



LEGEND:



Retained
Design rule carried
forward



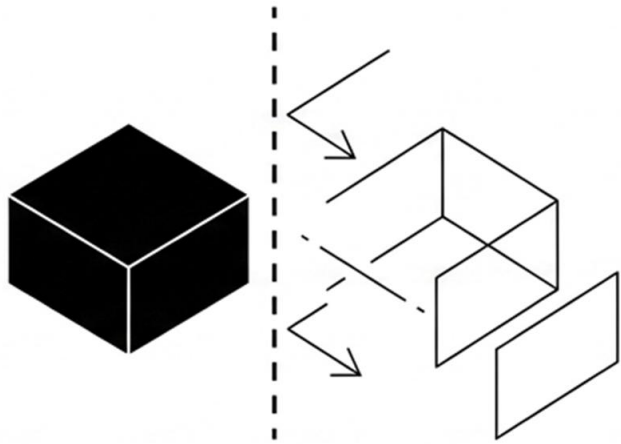
Partially retained
Design rule weakened or
not fully enforced



Lost
Design rule not carried
forward



Stripped
Design rule removed or
no longer tracked



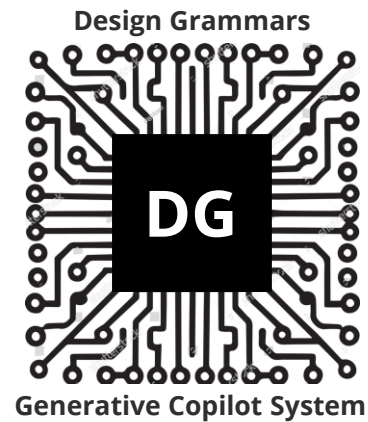
THE KNOWLEDGE GAP

- BIM automates information — but **design knowledge** stays **fragmented** across files, platforms and disciplines.
- Each handover **transfers only the final state** — the geometry and properties — **not the rules and constraints** that produced it.
- IFC (IFC4 / IFC4.3) is a product model for physical components and properties. It has **no first-class, executable representation of design rules** or parametric dependencies — IfcConstraint / IfcMetric are descriptive only.

The checkable-requirement stack — IDS, mvdXML, bSDD — verifies delivered models (presence and values). It **does not express the generative, parametric logic** that creates the geometry.

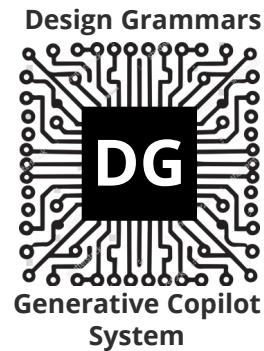
- **Parametric tools** (Grasshopper / Dynamo) silo that logic as opaque, **non-interoperable** node graphs that die at export — **the rules are replicated per platform, never shared.**

PROPOSED DESIGN KNOWLEDGE MANAGEMENT SYSTEM



THE DESIGN GRAMMARS

FROM COLLABORATIVE FILE-BASED EXCHANGE TO OBJECT-LEVEL KNOWLEDGE SHARING WITHIN INTEGRATED BIM

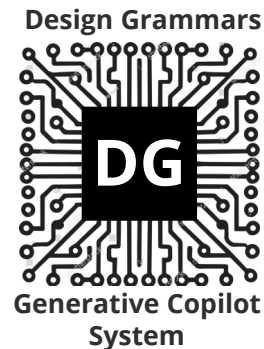


GOAL

Build a Shared Knowledge Core that carries design intent in a cross-platform environment of the BIM project and reuses it for validation and generation of design states during design development.

Example: "habitable rooms $\geq X$ m² with exterior daylight access" travels as a reusable rule — not as a frozen set of wall positions.

FROM COLLABORATIVE FILE-BASED EXCHANGE TO OBJECT-LEVEL KNOWLEDGE SHARING WITHIN INTEGRATED BIM



APPROACH

- Core–Periphery data model
- Abstract design objects + rules at the centre
- Domain-specific representations (BIM, parametric, analysis) at the edge
- Object-level semantic communication replaces file exchange
- Periphery = recognised views: BIM edge (IFC / authoring model), parametric edge (Grasshopper definition), analysis edge (energy / structural).
- Core = an ontology aligned to open vocabularies (BOT, Topologic, SOSA) — the Linked Building Data approach to one shared object model.

Where Design Grammars sits in the openBIM stack

DESIGN GRAMMARS complements the openBIM stack — it does not replace it. It carries the upstream design logic that IFC and IDS were never designed to hold, and can export IDS-checkable requirements downstream.

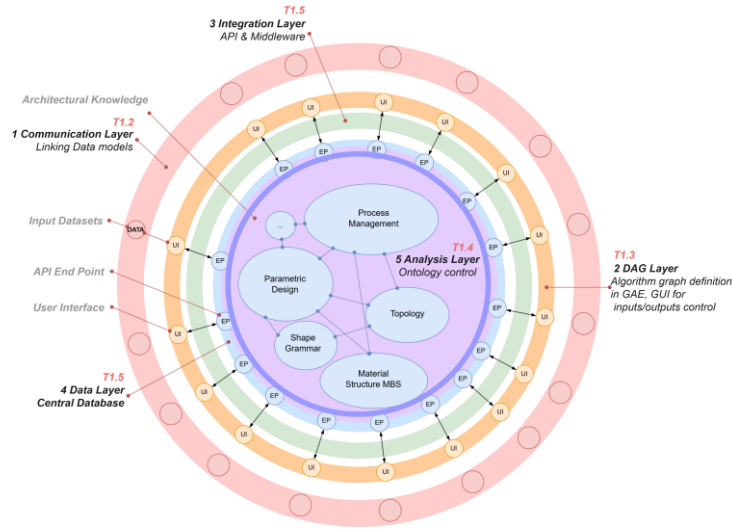
Concern	IFC4 / IFC4.3	IDS / mvdXML / bSDD	ISO 19650 / LOIN	Design Grammars
Physical components & properties	✓	—	governs	references
Machine-checkable delivered requirements	—	✓	specifies	exports to
Parametric / generative design logic	—	—	—	✓
Relational design rules (SWRL)	descriptive only	value / presence checks	—	✓ executable
Cross-tool object identity (linked data)	partial	—	—	✓ (ontology / IRIs)

RELATION TO STATE-OF-ART

Automated Code Compliance Checking (Solibri / SMC; Eastman et al. 2009; Beach et al.) — DG encodes rules from natural language into an open ontology via a constrained LLM, instead of hard-coded checkers.

Semantic rule checking / Linked Building Data (Pauwels et al.; BOT; SHACL) — DG adds the *generative* (grammar) side, not only checking.

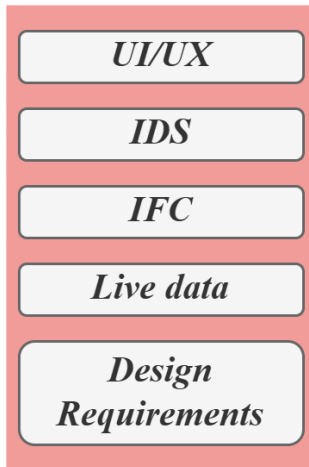
Shape / design grammars (Stiny 1980) — DG operationalises grammar rules inside a live BIM + parametric pipeline.



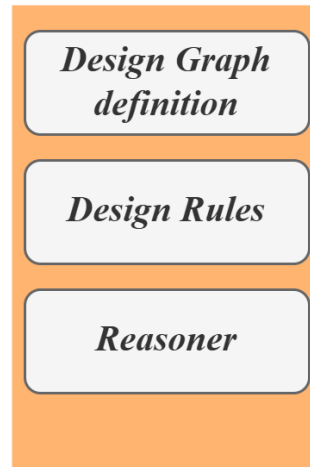
DG SYSTEM FRAMEWORK 5 LAYERS

Core-Periphery data model

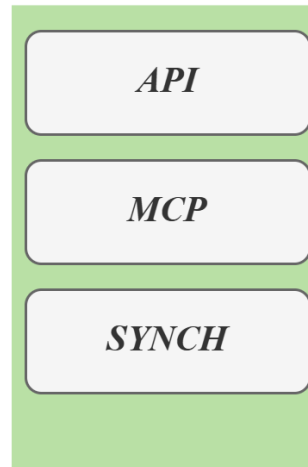
Communication



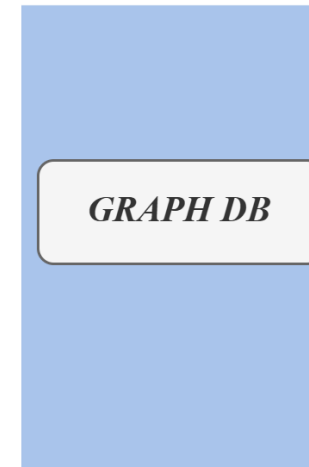
Computation



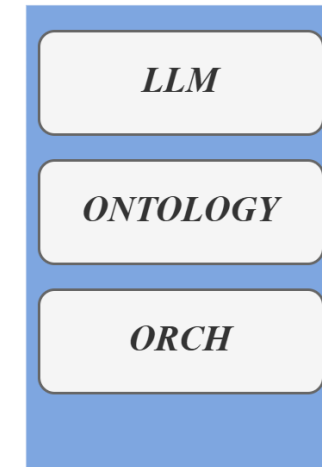
Integration



Data Core



Processing



DG ONTOLOGY

Class hierarchy (inferred)

Class hierarchy: ClassAtom

- owl:Thing
 - Feature
 - FeatureOfInterest
 - ObjectInstance
 - Geometry
 - GeoRef
 - GraphLayer
 - LayerEntity
 - KnowledgeGraph
 - KnowledgeClass
 - KnowledgeNote
 - KnowledgeSession
 - KnowledgeTag = skos:Concept
 - skos:Concept = KnowledgeTag
 - Metagraph
 - Atom = swrt:Atom
 - BuiltinAtom = swrt:BuiltinAtom
 - ClassAtom = swrt:ClassAtom**
 - DataPropertyAtom = swrt:DataValuedPropertyAtom
 - ObjectPropertyAtom = swrt:IndividualPropertyAtom
 - swrt:BuiltinAtom = BuiltinAtom
 - swrt:ClassAtom = ClassAtom
 - swrt:DataValuedPropertyAtom = DataPropertyAtom
 - swrt:IndividualPropertyAtom = ObjectPropertyAtom
 - Builtin = swrt:Builtin
 - DesignRuleSession
 - Literal
 - Rule = swrt:Imp
 - swrt:Atom = Atom
 - swrt:Builtin = Builtin
 - swrt:Imp = Rule
 - swrt:Variable = Variable
 - Variable = swrt:Variable
 - VariableScopeValue
 - OntoGraph
 - ValidationGraph
 - Observation
 - ValidationEntity
 - owl:Class
 - Class
 - owl:DatatypeProperty
 - DatatypeProperty
 - owl:ObjectProperty
 - ObjectProperty
 - prov:Activity
 - KnowledgeSession
 - ValidationRun
 - prov:Entity
 - ValidationEntity
 - skos:Concept = KnowledgeTag
 - swrt:Atom = Atom
 - swrt:Builtin = Builtin
 - swrt:BuiltinAtom = BuiltinAtom
 - swrt:ClassAtom = ClassAtom
 - swrt:DataValuedPropertyAtom = DataPropertyAtom
 - swrt:Imp = Rule
 - swrt:IndividualPropertyAtom = ObjectPropertyAtom
 - swrt:Variable = Variable
 - ValidationReport
 - ValidationRun
 - ValidationResult

Annotations: ClassAtom

Annotations +

rdfs:label [language: en] [in design-grammar]
ClassAtom

rdfs:comment [language: en] [in design-grammar]
An atom asserting class membership (e.g. Building(?b)). Takes one variable argument at pos 1.

graph [in design-grammar]
 [Metagraph](#)
 [Metagraph](#)

Description: ClassAtom

Equivalent To +
 **swrt:ClassAtom**

SubClass Of +
 **Atom**

General class axioms +

SubClass Of (Anonymous Ancestor)

-  **Atom**
-  **swrt:Atom**
-  **ClassAtom**

Instances +

-  **R_BUILDING_MAX_HEIGHT_75_V_A1**
-  **R_STREET_MIN_WIDTH_18_V_A1**

Target for Key +

Disjoint With +
 **BuiltinAtom, ObjectPropertyAtom, DataPropertyAtom**

Disjoint Union Of +

This mapping is organized across 4 functional layers:
OntoGraph, Metagraph, ValidationGraph, KnowledgeGraph.

OntoGraph Layer: OWL / RDFS

```

dg:Class ≡ owl:Class
dg:DatatypeProperty ≡ owl:DatatypeProperty
dg:ObjectProperty ≡ owl:ObjectProperty
dg:label ≡ rdfs:label
dg:range ≡ rdfs:range

```

Metagraph Layer: SWRL

```
dgm:Rule ≡ swrl:Imp (Implication)
dgm:ClassAtom ≡ swrl:ClassAtom
dgm:DataPropertyAtom ≡ swrl:DataValuedPropertyAtom
dgm:BuiltinAtom ≡ swrl:BuiltinAtom
dgm:Variable ≡ swrl:Variable
dgm:hasBody ≡ swrl:body
dgm:hasHead ≡ swrl:head
```

ValidationGraph Layer: PROV-O, SOSA, SHACL, GeoSPARQL

dgv:ValidationRun $\approx$ prov:Activity
dgv:ValidationEntity $\approx$ prov:Entity
dgv:ValidationRun $\approx$ sh:ValidationReport
dgv:ValidationEntity $\approx$ sh:ValidationResult
dgv:ObjectInstance $\approx$ geo:Feature
dgv:hasGeoRef $\equiv$ geo:hasGeometry

KnowledgeGraph Layer: SKOS + Dublin Core + SIOC

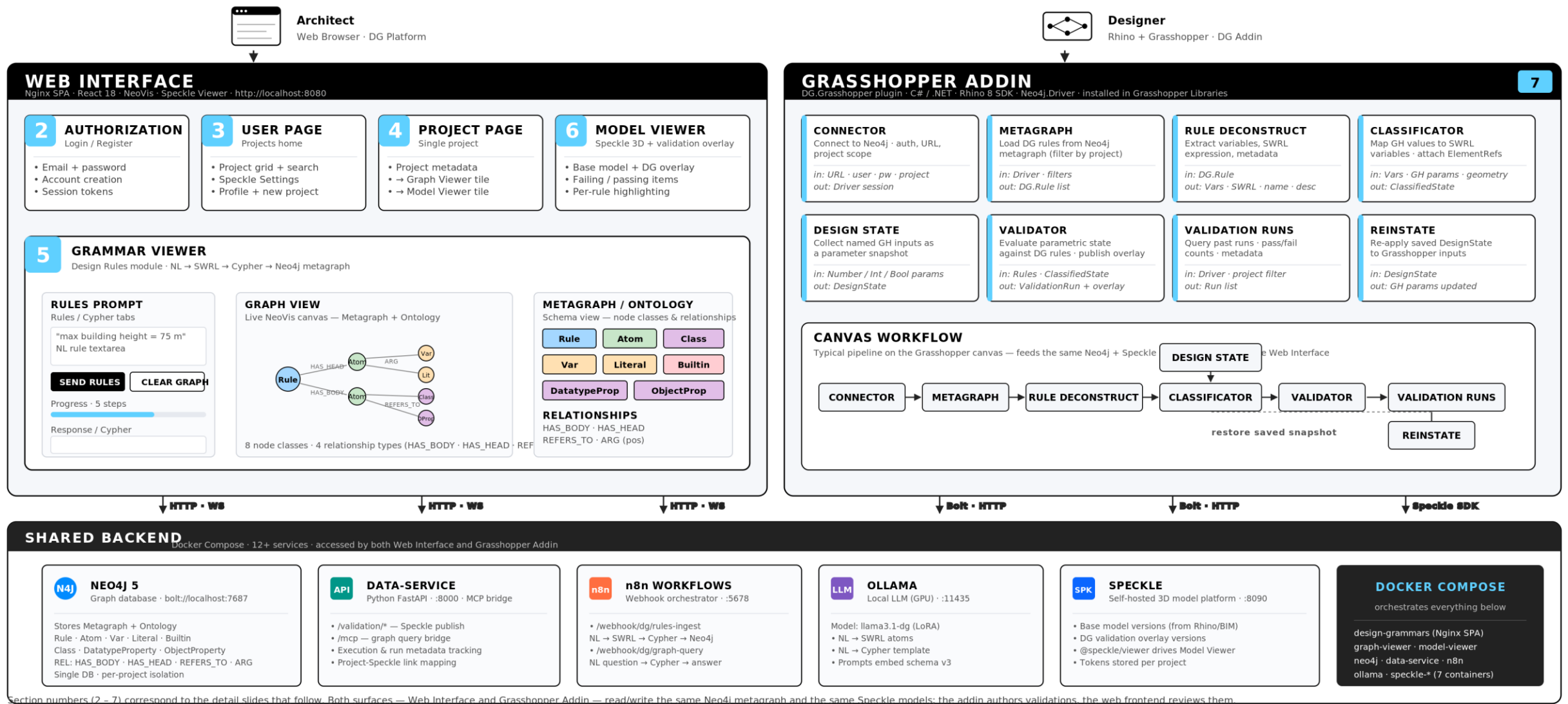
```

dkg:KnowledgeTag ≡ skos:Concept
dkg:KnowledgeClass ≈ skos:ConceptScheme
dkg:tagName ≡ skos:prefLabel
dkg:taggedWith ≈ skos:related

```


DG SYSTEM PROTOTYPE

ARCHITECTURE



DG SYSTEM PROTOTYPE

USE CASE. FROM CLIENT REQUIREMENTS TO VALIDATED DESIGN STATE

USE CASE — FROM CLIENT REQUIREMENTS TO VALIDATED DESIGN

Three stakeholders · four acts · Knowledge Graph → MetaGraph → ValidationGraph

1 CLIENT BRIEF

Requirements & constraints uploaded to DG

UPLOAD DESIGN BRIEF

Submit requirements & constraints, addressed to the Lead Architect.

NL TEXT INPUT

FILE UPLOAD

→ Transposed into the Knowledge Graph as Requirement / Constraint entities

2 RULE AUTHORIZING

NL prompts → SWRL → Cypher → MetaGraph

SYNTHESIZE RULES

NL prompts in the Grammar Viewer → LLM crafts SWRL atoms + Cypher.

NL → SWRL

SWRL → CYPHER

LLM ASSIST

→ Populates the MetaGraph; each Rule node is linked back to its source Requirement / Constraint

3 DESIGN BOOTSTRAP

Parametric inputs + DG objects generated

CONNECT DESIGN PLATFORM

Place DG Addin components in Grasshopper; DG bootstraps the parametric scaffold.

CONNECTOR

METAGRAPHCLASSIFICATOR + DESIGN STATE

→ DG emits initial parametric inputs & DG.Objects ready for interactive validation

4 VALIDATE & COMMUNICATE

Design states tracked · solution returned

REVIEW SOLUTION

Receives validated design proposal with explicit links to original brief.

SOLUTION REPORT

3D OVERLAY

→ Feeds revised requirements back into the Knowledge Graph (loop closes)

VALIDATE & COMMUNICATE

Reviews validation runs in Model Viewer; approves the chosen design state and composes the response to the Client.

RUN AUDIT

RULE-LEVEL DIFF

→ Knowledge Graph updated with outcome

RECORD & ITERATE STATES

Each design variant captured by VALIDATOR; changes tracked in the ValidationGraph with full provenance.

VALIDATOR

VALIDATION RUNS

REINSTATE

→ Failing / passing items pushed to Speckle

CLIENT
Provides brief
Receives result

- Domain expert
- Owns goals & constraints
- Non-technical user

LEAD ARCHITECT
Authors design rules
Validates outcome

- Curates requirements
- Drives rule synthesis
- Approves design states

COMPUTATIONAL DESIGNER
Wires design platform

- Owns parametric model
- Drives DG Addin
- Iterates design states

SYSTEM STATE · graph evolution: single source of truth; each act mutates a different sub-graph

KNOWLEDGE GRAPH

CREATE
Requirements & Constraints transposed from Client NL + file input.



METAGRAPH (SWRL)

POPULATE
Rule · Atom · Var · Literal · Builtin nodes; Linked back to the Knowledge Graph.



DESIGN SCAFFOLD

EMIT
Parametric inputs & DG.Objects derived from MetaGraph variables.



VALIDATION GRAPH + KNOWLEDGE GRAPH

TRACK · APPROVE · UPDATE
DesignState · ValidationRun nodes; outcome flows back to Knowledge Graph.



Knowledge Graph updates with validated decisions → Client review feeds the next iteration

DG SYSTEM PROTOTYPE

PHASE 1-2. Client Brief and Rule Authoring.

Web interface. Grammar Viewer

← Project

TestA

Grammar Viewer

Mode

▼

Insert Rules

Graph View

MetaGraph

Rules Prompt

Cypher Expression

Enter Design Rules in NL text

Send Rules

Clear the Graph

Send rules

LLM parse

Write graph

Annotate properties

Reload graph

Response

Workflow response will appear here...

Workflow Cypher

Cypher used to build the graph will appear here...

Node details

Rule

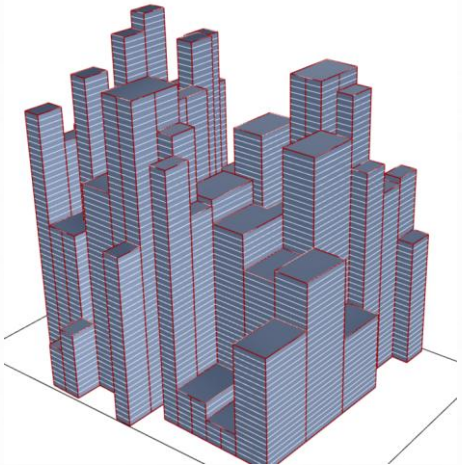
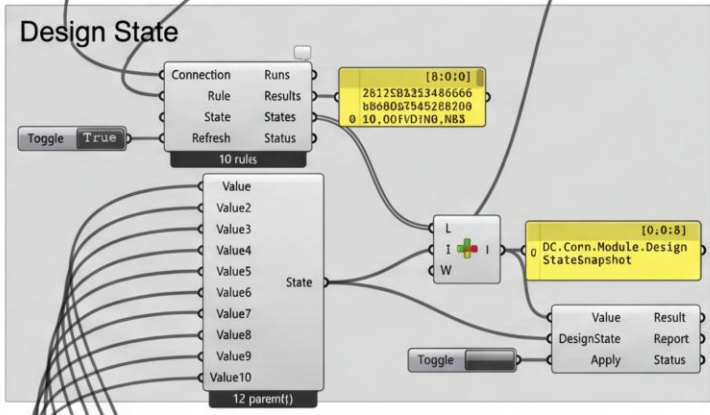
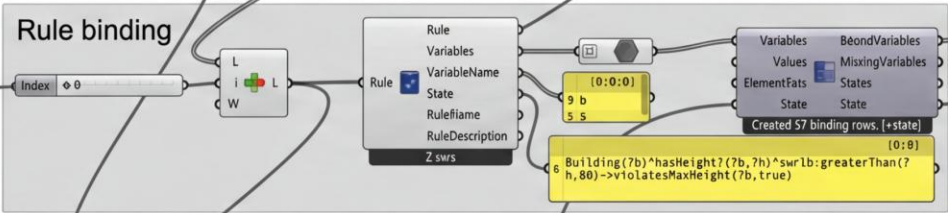
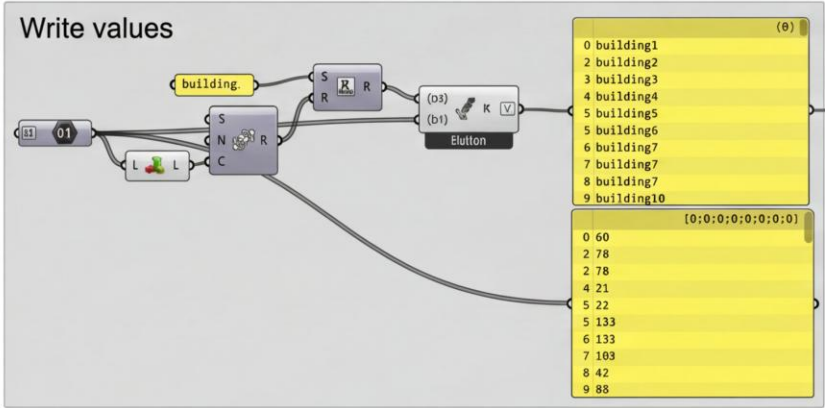
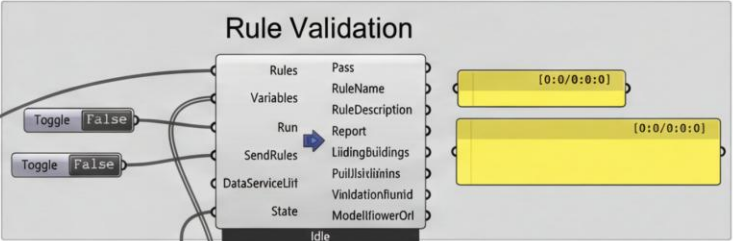
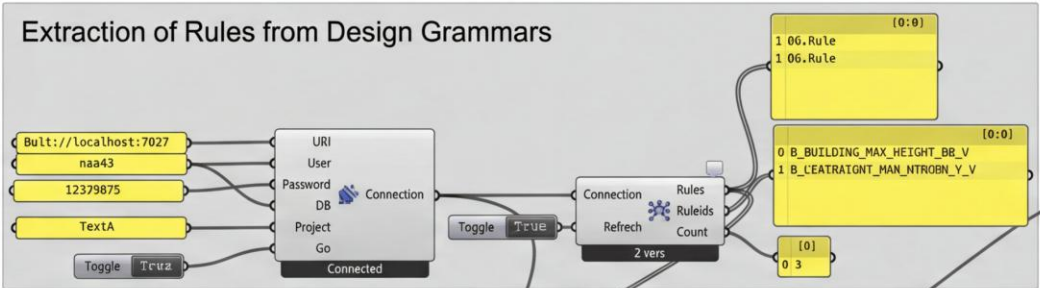
Key	Value
<id>	88
Rule_Id	"R_LIVINGUNIT_MIN_WINDOW_2_V"
description	"All living units must have at least 2 windows"
graph	"Metagraph"
kind	"violation"
project	"TestA"
text	"LivingUnit(?u)^hasWindowCount(?u,?w)^swrlb:lessThan(?w,2)->violatesMinWindowCount(?u,true)"

Add property

DG SYSTEM PROTOTYPE

PHASE 3. Design Bootstrap.

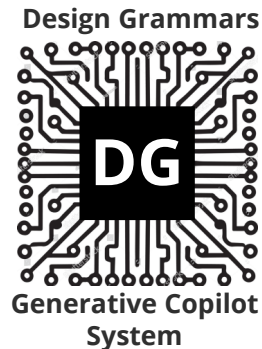
Grasshopper Addin



CONCLUSION

Contribution

A working prototype that encodes, manages and reuses grammar knowledge across the design lifecycle.



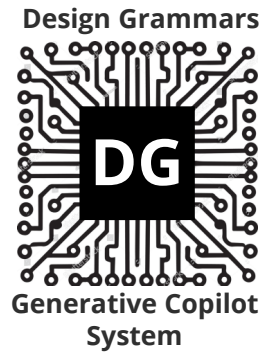
What works

- Plan-layout generation from a reusable spatial program
- Iterative validation of design states against rule sets
- Structured ingestion of project documentation

Evidence: R rules encoded $\cdot A$ SWRL atoms $\cdot$ encoding accuracy $P\%$ (precision) / $Q\%$ (recall) $\cdot$ case study of S spaces.

Limitations & next steps

- SWRL coverage limited to ClassAtom / DataPropertyAtom / BuiltinAtom — next: extend to full SWRL body coverage
- LLM Cypher output still needs human review — next: structured output + automated tests
- Academic evaluation only — production stress-testing pending
- IDS is a complement, not a gap: DG can export IDS-checkable requirements — verification downstream, generative logic upstream. Full IDS round-trip is the next step.



**THANK YOU
FOR YOUR
ATTENTION**

REFERENCES

- [1] Hubka, V., and W. E. Eder. 2002. "Theory of technical systems and engineering design synthesis." In Engineering design synthesis: Understanding, approaches and tools (pp. 49-66). London: Springer London.
- [2] Lawson, B. (2006). How designers think.
- [3] J. Poon, M.L. Maher, Co-evolution and emergence in design, Artif. Intell. Eng. 11 (3) (1997) 319–327.
- [4] D.A. Schon, Designing as reflective conversation with the materials of a design situation, Res. Eng. Des. 3 (3) (1992) 131–147.
- [5] Cross, N. "Analysing design activity." (1996).
- [6] K. Dorst, N. Cross, Creativity in design process: co-evolution of problem-solution, Des. Stud. 22 (2001) 425–437.
- [7] Gero, J. S. 1992. "What's in a design case and how it can be used." AID'92 Workshop on Case Based Design Systems, pp. 27-28.
- [8] K. Mubarak, Case based reasoning for design composition in architecture, Carnegie Mellon University, Pittsburgh, 2004.
- [9] Fu, K., J. Murphy, M. Yang, K. Otto, D. Jensen, and K. Wood. 2015. "Design-by- analogy: experimental evaluation of a functional analogy search methodology for concept generation improvement." Res Eng Des. Springer London
- [10] Gero, J. S. 1990. "Design prototypes. A knowledge representation schema for design," AI Mag., vol. 11, no. 4, pp. 26–36, 1990.
- [11] K. Dorst, The core of 'design thinking' and its application, Des. Stud. 32 (6) (2011) 521–532.
- [12] Lin, Chieh Jen 'Design Criteria Modeling - Use of Ontology-Based Algorithmic Modeling'
- [13] Prof. Dr. Harald Sack, OpenHPI - Course Knowledge Engineering with Semantic Web Technologies: Ontological Engineering, 2019
- [14] Mads Holten Rasmussen, BOT: The building topology ontology of the W3C linked building data group
- [15] <https://topologic.app/tutorials/Topologic-01-Introduction.pdf>
- [16] Aish, R., Jabi, W., Lannon, S., Wardhana, N., & Chatzivasileiadi, A. (2018). Topologic: tools to explore architectural topology.
- [17] Rasmussen, M. H., Lefrançois, M., Schneider, G. F., Pauwels, P., & Janowicz, K. (2020). BOT: the Building Topology Ontology of the W3C Linked Building Data Group.
- [18] Lin, C. J. (2016). Design Criteria Modeling - Use of Ontology-Based Algorithmic Modeling to Represent Architectural Design Criteria at the Conceptual Design Stage.
- [19] Prof. Dr. Harald Sack. (2023, November 15). (1337) Knowledge Graphs - Excursion 7 The Semantic Web Rule Language SWRL
- [20] Succar, B. (2016). *BIMe Initiative Model Uses List*. <http://bit.ly/BIMPaperA10>
- [21] Peffers, K., Tuunanen, T., & Rossi, M. (2006). *The design science research process: A model for producing and presenting information systems research*.
- [22] Zhu, J., Wu, P., & Lei, X. (2023). IFC-graph for facilitating building information access and query. Automation in Construction, 148.
- [23] Bobed, C., Maillot, P., Cellier, P., & Ferré, S. (2020). Data-driven assessment of structural evolution of RDF graphs. Semantic Web, 11(5), 831–853.
- [24] Building performance optimisation: A hybrid architecture for the integration of contextual information and time-series data. (2016). Automation in Construction, 70, 51–61.
- [25] buildingSMART. Information Delivery Specification (IDS) — technical documentation; mvdXML; bSDD.
- [26] ISO 16739-1 (IFC4.3); ISO 19650-1/-2 (information management); LOIN / ISO 7817.
- [27] Eastman, C., Lee, J., Jeong, Y., Lee, J. (2009). Automatic rule-based checking of building designs. Automation in Construction 18(8), 1011–1033.
- [28] Pauwels, P., Zhang, S., Lee, Y.-C. (2017). Semantic web technologies in AEC industry: A literature overview. Automation in Construction 73, 145–165.
- [29] Beach, T., Rezgui, Y., et al. Building compliance / regulatory knowledge representation for BIM.
- [30] Stiny, G. (1980). Introduction to shape and shape grammars. Environment and Planning B 7(3), 343–351.
- [31] Solibri Model Checker (SMC) — commercial automated code-compliance baseline.